

Information Retrieval Project Report

Satish Vepuri

1 Introduction

This project focuses on building the retrieval component of an information retrieval code using the Vector Space Model. The project code uses TF-IDF weighting and cosine similarity to rank documents based on their relevance to a query. Queries were extracted from the `topics.txt` file, which provided three different sections for each query, namely, title, description, and narrative. I evaluated my algorithm using these three configurations to observe how the structure and length of a query influence retrieval performance. Precision and recall values were computed using the provided judgments.

2 Methodology

2.1 Data Structures

I used STL containers for most of the code since they give fast lookups and are simple to manage. The main ones I relied on were `unordered_map` and `unordered_set`. These containers are helpful because the code needs dictionary-style access for terms and documents many times during retrieval.

Term Dictionary

```
unordered_map<string, int> word_index;
```

Stores each processed (stemmed) term and assigns it a numeric ID.

Document Dictionary

```
unordered_map<int, string> doc_index;
```

Maps internal document IDs back to document names that appear in the `qrels` file.

Forward Index

```
unordered_map<int, unordered_map<int, int>> forward_index;
```

Each document maintains a map of term IDs and their raw frequencies. This structure is later used to construct document vectors.

Inverted Index

```
unordered_map<int, unordered_map<int, int>> inverted_index;
```

Each term ID maps to all documents in which it appears. This structure enables efficient retrieval at query time.

Relevance Judgments

```
unordered_map<int, unordered_set<string>> relevance;
```

Each query ID stores a set of relevant document names for performance evaluation.

Stopword List

```
unordered_set<string> stopwords;
```

Common English words are removed during tokenization and before stemming.

These STL-based structures allowed me to build the indexing and retrieval logic without needing additional custom classes.

2.2 Term Weighting and Normalization

TF-IDF weighting is used for both document vectors and query vectors. The IDF values follow:

$$idf(t) = \log \left(\frac{N + 1}{df(t) + 1} \right) + 1$$

where:

- N is the total number of documents, and
- $df(t)$ is the number of documents that has term t .

Term frequency is scaled as:

$$tf_w = 1 + \log(tf)$$

The final TF-IDF term weight is:

$$w_t = (1 + \log(tf)) \cdot idf(t)$$

Cosine similarity is used to compare query and document vectors:

$$sim(q, d) = \frac{q \cdot d}{\|q\| \|d\|}$$

Only terms that appear in the query contribute to the dot product, which speeds up scoring significantly.

2.3 Query Processing

Queries are read from the `topics.txt` file. I tracked the active XML tag (`<title>`, `<desc>`, or `<narr>`), and appended text until reaching the `</top>` tag.

For each query, I created three separate query versions:

1. title
2. title + description
3. title + narrative

Tokenization and Stemming

The updated query pipeline performs the following steps:

- convert text to lowercase
- remove non-alphabetic characters
- remove stopwords
- apply the Porter Stemmer to each remaining term

Stemming reduces word variants to a common root, increasing the likelihood that query terms match document terms.

2.4 Retrieval Algorithm

Retrieval follows the vector space scoring method, with modifications due to the new query-processing pipeline.

The full retrieval steps are:

1. tokenize and stem the query
2. construct the query TF-IDF vector
3. gather candidate documents:
 - for each stemmed query term, look up its posting list
 - insert document IDs into an `unordered_set` to avoid duplicates
4. for each candidate document:
 - compute its TF-IDF vector on the fly
 - compute cosine similarity
5. sort all scored documents in descending similarity
6. write the top 1000 documents to the output file for that query setting

This avoids scanning the entire collection and evaluates only documents sharing at least one term with the query.

2.5 Performance Evaluation

To evaluate performance, I compared the ranked list of retrieved documents with the relevance judgments. For each query setting, I computed:

$$precision = \frac{\text{relevant retrieved}}{\text{retrieved}}$$

$$recall = \frac{\text{relevant retrieved}}{\text{total relevant}}$$

These values were written to the performance file along with the query ID and the setting used.

3 Results and Discussion

The precision and recall values are shown in Table 1.

The precision and recall values for the four queries shows a consistent pattern where the title-only formulation tends to produce higher precision, and using the description or the narrative usually raises recall. In Query 353, we see that the title form gives a precision of 0.0543 with a recall of 0.3182. But, when the description and narrative are added, the

QueryID	Setting	Precision	Recall
352	title	0.0020	0.0385
352	title+desc	0.0010	0.0192
352	title+narr	0.0020	0.0385
353	title	0.0543	0.3182
353	title+desc	0.0080	0.3636
353	title+narr	0.0100	0.4545
354	title	0.0155	0.2800
354	title+desc	0.0080	0.3200
354	title+narr	0.0080	0.3200
359	title	0.0011	0.1667
359	title+desc	0.0010	0.1667
359	title+narr	0.0010	0.1667

Table 1: Precision and Recall values for all queries and query field settings.

precision drops to 0.0080 and 0.0100 and recall reaches 0.3636 and 0.4545. As more text (description/narrative) brings in many more terms, which lead the code to match a wider range of documents, and some of these happen to be relevant even though the ranking quality becomes weaker.

Query 354 behaves in a similar way, although the differences feel a bit smaller. The title-only precision is 0.0155, and both extended forms settle at 0.0080, and recall increases to 0.3200. In Query 352 the code struggles to identify strong matches, so both precision and recall remain low regardless of which parts of the query text are used. Query 359 ends up showing almost identical numbers across all three settings, which suggests the important vocabulary appears early in the topic statement and the rest of the query does not introduce much that would change retrieval behavior.

Across all queries, the title text usually stays the most focused and avoids the extra noise that longer fields introduce. Adding the description or narrative tends to expand the query enough to pick up additional relevant documents, but it also increases overlap with unrelated material, so precision sinks and recall improves. This tradeoff shows up clearly in the pattern of the results.

4 Conclusion

The code was able to use the earlier project’s data files to compute TF-IDF weights, construct document and query vectors, and rank documents with cosine similarity. The evaluation demonstrated how different query formulations influence retrieval effectiveness. Title-only queries produced the best precision in most cases, and description and narrative expansions increased recall at the cost of precision. These trends are consistent with expected TF-IDF

behavior when working with natural-language queries.

The results were stored in `satish_title.txt`, `satish_title_desc.txt`, `satish_title_narr.txt` and `satish_performance.txt` files.